The PSK client is very similar to the server, as shown in listing 12.11. As before, you create a new `BcTlsCrypto` object and use that to initialize a `PSKTlsClient` object. In this case, you pass in the PSK and an identifier for it. If you don't have a good identifier for your PSK already, then a secure hash of the PSK works well. You can use the `Crypto.hash()` method from the Salty Coffee library from chapter 6, which uses SHA-512. As for the server, you need to override the `getSupportedVersions()` method to ensure DTLS support is enabled. You can then connect to the server and perform the DTLS handshake using the `DTLSClientProtocol` object. The `connect()` method returns a `DTLSTransport` object that you can then use to send and receive encrypted packets with the server.

Create a new file named PskClient.java alongside the server class and type in the contents of the listing to create the server. If your editor doesn't automatically add them, you'll need to add the following imports to the top of the file:

```
import static java.nio.charset.StandardCharsets.UTF_8;
import java.io.FileInputStream;
import java.net.*;
import java.security.*;
import org.bouncycastle.tls.*;
import org.bouncycastle.tls.crypto.impl.bc.BcTlsCrypto;
```

Listing 12.11 The PSK client

```
package com.manning.apisecurityinaction;
public class PskClient {
    public static void main(String[] args) throws Exception {
```